

ModuleAutoDialerManage User Guide

Bit Dream IT

- [ModuleAutoDialerManage — User Guide](#)
 - [Table of Contents](#)
 - [1. What's new in this Bit Dream IT edition](#)
 - [2. Installation](#)
 - [Requirements](#)
 - [Install steps](#)
 - [Uninstall — keep your data](#)
 - [3. Quickstart — your first campaign in 5 minutes](#)
 - [Step 1 — Set up an extension to receive calls](#)
 - [Step 2 — Open the module](#)
 - [Step 3 — Configure module settings \(top of page\)](#)
 - [Step 4 — Add an extension mapping](#)
 - [Step 5 — Create a campaign via REST API](#)
 - [Step 6 — Watch it dial](#)
 - [4. The main module page](#)
 - [Module settings reference](#)
 - [5. Creating a campaign](#)
 - [Via the UI \(recommended for operators\)](#)
 - [Via REST API \(recommended for automation\)](#)
 - [Field reference](#)
 - [Number object](#)
 - [Updating a campaign](#)
 - [Deleting a campaign](#)
 - [CSV bulk import](#)
 - [6. Campaigns list page](#)
 - [Filter bar \(top\)](#)
 - [Sortable columns](#)
 - [Per-row actions \(right column\)](#)
 - [New campaign button \(top right\)](#)
 - [Live dashboard button \(top right\)](#)
 - [7. Working with surveys \(polls\)](#)
 - [Creating a survey](#)

- Audio files
- Polling results
- 7. Extensions setup
- 8. Live dashboard
 - Overview mode (default)
 - Detail mode (drill-down)
 - Auto-refresh
- 9. DNC blacklist
 - Adding numbers
 - Listing and searching
 - Sources
 - Effect on dialer
- 10. AMD (Answering Machine Detection)
 - Enabling AMD
 - How it works
 - Tuning AMD parameters
 - Rebuild dialplan after change
- 11. Scheduling (business hours)
 - Time-of-day: `timeStart` and `timeEnd`
 - Day-of-week: `scheduleDays`
 - Example: Mon-Fri 09:00-18:00
 - How the dialer enforces this
- 12. Webhook — get notified when a campaign state changes
 - Event 1: `campaign.state_changed`
 - Event 2: `campaign.completed`
 - Setting the webhook URL
 - Webhook delivery guarantees
 - Sample receiver (Laravel)
- 13. Call recordings
 - Response
- 14. Reports & CSV export
 - Campaign summary
 - CSV export
 - Live results sync (incremental)
- 15. Settings reference
 - Tasks model fields

- [Blacklist model fields](#)
- [16. Troubleshooting](#)
 - [Module won't install — “ParseError” or “Class not found”](#)
 - [Campaign is “open” but no calls are being made](#)
 - [Worker not running at all](#)
 - [Dashboard shows no campaigns](#)
 - [Webhook not firing](#)
 - [AMD is detecting humans as machines](#)
 - [Can't access dashboard page \(404\)](#)
- [17. Uninstall & rollback](#)
 - [Uninstall the module \(keep data\)](#)
 - [Roll back to original `ModuleAutoDialer`](#)
 - [Permanently delete all module data](#)
- [Support](#)

ModuleAutoDialerManage — User Guide

Version 1.35 (Bit Dream IT edition) Module unique ID: `ModuleAutoDialerManage` **Upstream:** `mikopbx/ModuleAutoDialer` v1.35 by Alexey Portnov & Nikolay Beketov **License:** GPL-3.0-or-later

This module turns MikoPBX into a professional outbound campaign dialer. It can call a list of phone numbers, play an IVR poll, connect answered calls to your agents, and report results to an external CRM via REST API.

Table of Contents

- [1. What's new in this Bit Dream IT edition](#)
- [2. Installation](#)
- [3. Quickstart — your first campaign in 5 minutes](#)
- [4. The main module page](#)
- [5. **Creating a campaign via UI**](#)
- [6. **Campaigns list page**](#)
- [7. Working with surveys \(polls\)](#)
- [8. Extensions setup](#)
- [9. Live dashboard](#)
- [10. Call results browser](#)
- [11. IVR / poll answers browser](#)

12. [Audio files management](#)
 13. [DNC blacklist](#)
 14. [AMD \(Answering Machine Detection\)](#)
 15. [Scheduling \(business hours\)](#)
 16. [Webhook — get notified when a campaign state changes](#)
 17. [Call recordings](#)
 18. [Reports & CSV export](#)
 19. [Settings reference](#)
 20. [Troubleshooting](#)
 21. [Uninstall & rollback](#)
-

1. What's new in this Bit Dream IT edition

This is a fork of MIKO's `ModuleAutoDialer` v1.35, with the following new features added by Bit Dream IT:

Feature	What it does
Live dashboard	Real-time campaign monitoring with progress bars, in-progress call counts, agent status grid, and recent call feed. Auto-refreshes every 4 seconds.
DNC (Do-Not-Call) blacklist	A separate list of phone numbers that the dialer will NEVER call, regardless of which campaign they appear in. Manage via UI or REST API.
AMD (Answering Machine Detection)	Per-campaign toggle. When enabled, Asterisk's <code>AMD()</code> app runs on the customer leg before bridging. Voicemail machines are hung up automatically — saving agent time.
Webhook on completion	Each campaign can have a <code>callbackUrl</code> . When all numbers are dialed, the module POSTs <code>{event: "campaign.completed", task_id, total, answered, ...}</code> to that URL.
Scheduling (business hours)	New <code>scheduleDays</code> field on each campaign. Comma-separated ISO weekday numbers (1=Mon ... 7=Sun). The dialer skips days outside the schedule.
Campaign summary report	One-click totals: total dialed, answered, answer rate %, average duration, total duration.
CSV export	Download all results of a campaign as a CSV file with one click.
CSV import	Bulk-upload phone numbers from a CSV file with optional name and personalization params per number.
Test call	Dial a single test number to preview a poll before launching the full campaign.
Recording lookup	REST endpoint that joins campaign call records to MikoPBX's CDR table to find the audio recording file for any call.
Coexistence with original module	All DB tables, dialplan contexts, AMI user, REST API URL prefix, spool file names, and Asterisk sound identifiers have been renamed — so this module can run alongside the original <code>ModuleAutoDialer</code> on the same PBX without conflict.
Data migration	On install, all your existing data from <code>m_ModuleAutoDialer</code> , <code>m_Clients</code> , <code>m_Tasks</code> , <code>m_Polling</code> , etc. is copied into the new prefixed tables. Old tables are NOT dropped, so you keep your data as a backup.

2. Installation

Requirements

- MikoPBX version `2024.1.114` or newer
- PHP 7.4+ (PHP 8.x supported)
- Asterisk with `app_amd` loaded (for AMD feature — already bundled with MikoPBX)
- (Optional) `ModuleRHVoice` for free text-to-speech in Russian
- (Optional) Yandex Cloud API key for premium TTS/STT

Install steps

1. **Back up MikoPBX** — System → Backup → Create backup.
2. Download `ModuleAutoDialerManage.zip`.
3. MikoPBX → **Modules** → **Install from file** → upload the zip.
4. After install, check the module's message log. You should see lines like:

```
Migrated N rows (skipped 0 duplicates) from m_ModuleAutoDialer to  
m_ModuleAutoDialerManage  
Migrated N rows (skipped 0 duplicates) from m_Clients to  
m_ModuleAutoDialerManage_Clients
```

5. Clear volt cache: `rm -rf /var/tmp/www_cache/volt/* /var/tmp/www_cache/translations/*` and restart PHP-FPM: `/etc/rc.d/rc.php-fpm restart`
6. Restart module workers: `/etc/rc.d/rc.worker-safe-scripts restart`
7. The sidebar should now show three new items under “Routing”:
 - **Auto dialer** (main module page)
 - **Dialer dashboard** (live monitoring)
 - **DNC blacklist** (do-not-call list)

Uninstall — keep your data

When you uninstall the module via MikoPBX UI, the module's tables are KEPT by default (unless you check “Delete settings”). This is a safety feature so you can re-install without losing campaigns.

To permanently delete all data, run this SQL after uninstall:

```

DROP TABLE IF EXISTS m_ModuleAutoDialerManage_Blacklist;
DROP TABLE IF EXISTS m_ModuleAutoDialerManage_ClientsPhones;
DROP TABLE IF EXISTS m_ModuleAutoDialerManage_ClientsProperties;
DROP TABLE IF EXISTS m_ModuleAutoDialerManage_Clients;
DROP TABLE IF EXISTS m_ModuleAutoDialerManage_TaskResults;
DROP TABLE IF EXISTS m_ModuleAutoDialerManage_PollingResults;
DROP TABLE IF EXISTS m_ModuleAutoDialerManage_QuestionActions;
DROP TABLE IF EXISTS m_ModuleAutoDialerManage_Question;
DROP TABLE IF EXISTS m_ModuleAutoDialerManage_Polling;
DROP TABLE IF EXISTS m_ModuleAutoDialerManage_Tasks;
DROP TABLE IF EXISTS m_ModuleAutoDialerManage_DialerExtensions;
DROP TABLE IF EXISTS m_ModuleAutoDialerManage_AudioFiles;
DROP TABLE IF EXISTS m_ModuleAutoDialerManage;

```

3. Quickstart — your first campaign in 5 minutes

This is the fastest path from install to first call. Detailed options are explained in later sections.

Step 1 — Set up an extension to receive calls

The dialer needs an internal extension to bridge answered calls to. If you don't have one yet: 1. MikoPBX → **Extensions** → **Add extension** 2. Create extension **200** (or any free number), give it a name like "Dialer agent" 3. Save

Step 2 — Open the module

MikoPBX sidebar → **Auto dialer** (under Routing group).

Step 3 — Configure module settings (top of page)

Field	What to enter
Yandex API key	Your Yandex Cloud IAM token (optional, only needed for Yandex TTS)
Default dial prefix	999 (or whatever prefix your SIP trunk requires for outbound)
TTS service	Choose Yandex or RHVoice

Click **Save**.

Step 4 — Add an extension mapping

Scroll to the **Extensions** table. Click **Add**: - **Extension**: 200 (the one you created in step 1) - **Survey if new client**: leave empty for now - **Survey if known client**: leave empty for now

Save. This tells the dialer: “When a call is answered, bridge it to extension 200.”

Step 5 — Create a campaign via REST API

Open a terminal on the PBX (or any machine that can reach the PBX). Run:

```
curl -X POST http://YOUR_PBX_IP/pbxcore/api/module-dialer-manage/v1/task \
-H "Content-Type: application/json" \
-d '{
  "crmId": 1,
  "name": "Quickstart test",
  "state": 0,
  "innerNum": "200",
  "innerNumType": "exten",
  "maxCountChannels": 1,
  "dialPrefix": "999",
  "numbers": [
    {"number": "7912345678", "params": {"speech": "Hello, this is a test call."}}
  ]
}'
```

(Replace **YOUR_PBX_IP** and the test phone number with your own.)

Step 6 — Watch it dial

Open the **Dialer dashboard** in the sidebar. You should see: - The “Quickstart test” campaign card with state “open” - A progress bar - The in-progress count go to 1 when the call starts - The recent calls feed update with the call result

That's it — you've made your first campaign call. The next sections explain each feature in detail.

4. The main module page

The main page (sidebar → **Auto dialer**) has four sections:

1. **Module settings** — TTS provider, API keys, default dial prefix
2. **Extensions table** — Internal extensions the dialer can bridge calls to
3. **Surveys table** — IVR polls that can be played to callers
4. **CRM connection** (optional) — URL/login/password for syncing with 1C or external CRM

Module settings reference

Field	Description
Yandex API Key	Yandex Cloud IAM token for Yandex SpeechKit TTS/STT
Yandex Folder ID	Yandex Cloud folder ID (for STT billing)
TTS service	Yandex or RHVoice . RHVoice is free, Russian-only, lower quality
Default dial prefix	Prefix prepended to every outbound number (e.g. 999 to route through a specific trunk)
Confirm STT	If checked, the system reads back the recognized speech for confirmation
Need recognize	Enable speech recognition (Yandex STT) on poll answers

5. Creating a campaign

A **campaign** (called a “task” in the API) is a list of phone numbers to dial, plus rules about how to dial them.

Via the UI (recommended for operators)

1. Sidebar → **Campaigns** (under Routing group)
2. Click **New campaign** button (top right)
3. Fill in the form:
 - **Basic section** — name, CRM ID, state (open/paused/closed), description
 - **Call routing** — type (extension or survey), inner number/extension, max simultaneous calls, dial prefix, callback mode
 - **Schedule** — start time, end time, allowed weekdays (multi-select)
 - **Retry policy** — max attempts, seconds between retries, “retry until signal” toggle
 - **Advanced** — AMD toggle, webhook URL
 - **Phone numbers** — add via 3 tabs:
 - **Single add** — number + optional name + TTS text
 - **Paste bulk** — paste multi-line list (format: **number, name, params**)
 - **CSV upload** — upload CSV file with header **number, name, params**
4. Click **Create campaign** — you’ll be redirected to the edit page

Via REST API (recommended for automation)

```
POST /pbxcore/api/module-dialer-manage/v1/task
Content-Type: application/json
```

```
{
  "crmId": 1001,
  "name": "Payment reminders - June",
  "description": "Reminder campaign for outstanding June invoices",
  "state": 0,
  "innerNum": "200",
  "innerNumType": "exten",
  "maxCountChannels": 5,
  "dialPrefix": "999",
  "timeStart": 540,
  "timeEnd": 1080,
  "scheduleDays": "1,2,3,4,5",
  "maxAttempt": 3,
  "tryInterval": 300,
  "amdEnabled": 1,
  "callbackUrl": "https://crm.example.com/webhooks/dialer",
  "numbers": [
    {"number": "7912345678", "params": {"speech": "Your balance is 1000 rubles."}},
    {"number": "7912345679", "params": {"speech": "Your balance is 500 rubles."}}
  ]
}
```

Field reference

Field	Type	Required	Description
crmId	string	yes	External ID from your CRM (any unique string)
name	string	yes	Human-readable campaign name
description	string	no	Optional description shown on dashboard
state	int	yes	0 = open (active), 1 = closed, 2 = paused
innerNum	string	yes	Extension number to bridge answered calls to
innerNumType	string	yes	exten (bridge to agent) or polling (play IVR poll)
maxCountChannels	int	yes	Max simultaneous calls for this campaign
dialPrefix	string	no	Prefix prepended to every outbound number
timeStart	int	no	Minutes from midnight (e.g. 540 = 09:00)
timeEnd	int	no	Minutes from midnight (e.g. 1080 = 18:00)
scheduleDays	string	no	Comma-separated ISO weekday numbers (1=Mon ... 7=Sun). Empty = all days
maxAttempt	int	no	Max retry attempts per number (default 1)
tryInterval	int	no	Seconds between retries (default 60)
amdEnabled	int	no	1 = enable Answering Machine Detection, 0 = off
callbackUrl	string	no	Webhook URL called when campaign completes
numbers	array	yes	Array of {number, params} objects

Number object

```
{
  "number": "7912345678",
  "params": {
    "speech": "Your balance is 1000 rubles.",
    "custom_field": "any_value"
  }
}
```

- **number** — phone number, any format (non-digits stripped automatically)
- **params** — arbitrary JSON object. The **speech** field is read aloud by TTS. Other fields are passed to your CRM webhook.

Updating a campaign

```
PUT /pbxcore/api/module-dialer-manage/v1/task/42
Content-Type: application/json

{"state": 2} # pause the campaign
```

Common update use cases: - Pause: `{"state": 2}` - Resume: `{"state": 0}` - Stop and close: `{"state": 1}` - Increase concurrency: `{"maxCountChannels": 10}` - Enable AMD on the fly: `{"amdEnabled": 1}`

Deleting a campaign

```
DELETE /pbxcore/api/module-dialer-manage/v1/task/42
```

This deletes the campaign AND all its call results. The numbers are NOT removed from the **m_Clients** table (so you can re-add them to another campaign).

CSV bulk import

Instead of listing numbers in the JSON body, you can upload a CSV file:

```
curl -X POST http://YOUR_PBX_IP/pbxcore/api/module-dialer-manage/v1/task/42/import-csv \
-F "file=@numbers.csv"
```

CSV format (header row required):

```
number,name,params
7912345678,Ivan Ivanov,"{"speech":"Your balance is 1000"}"
7912345679,Petr Petrov,"{"speech":"Your balance is 500"}"
```

- **number** (required) — phone number
- **name** (optional) — caller name
- **params** (optional) — JSON string OR plain text (becomes `{text: "..."}`)

6. Campaigns list page

Sidebar → **Campaigns** (under Routing group)

This is the main management page for all your campaigns. Features:

Filter bar (top)

- **Search by name** — full-text search on campaign name + description + ID
- **State dropdown** — filter by open / closed / paused
- **Type dropdown** — filter by extension-bridged or survey/IVR
- **Search button** — apply filters

Sortable columns

Click any column header to sort ascending/descending: - ID, Name, State, Type, Inner num, Channels, Time window, Schedule, Flags

Per-row actions (right column)

Each campaign row has 5 action buttons: -  **Live detail** — open the dashboard detail page for this campaign -  **Edit** — open the campaign form to edit settings/numbers -  **CSV** — download all call results for this campaign as CSV -  **Pause/Resume** — toggle state between open (0) and paused (2) -  **Delete** — permanently delete the campaign (asks for confirmation)

New campaign button (top right)

Click **New campaign** to open the create form (see §5).

Live dashboard button (top right)

Click **Live dashboard** to open the dashboard overview.

7. Working with surveys (polls)

A **survey** (poll) is a multi-question IVR tree. When `innerNumType=polling`, the dialer plays the survey instead of bridging to an agent.

Creating a survey

```
POST /pbxcore/api/module-dialer-manage/v1/polling
Content-Type: application/json

{
  "name": "Customer satisfaction",
  "questions": [
    {
      "id": 1,
      "text": "How would you rate our service? Press 1 for excellent, 2 for good, 3 for poor.",
      "lang": "ru",
      "actions": [
        {"key": "1", "value": "excellent", "nextQuestion": 2},
        {"key": "2", "value": "good", "nextQuestion": 2},
        {"key": "3", "value": "poor", "nextQuestion": 2}
      ]
    },
    {
      "id": 2,
      "text": "Would you recommend us to friends? Press 1 for yes, 2 for no.",
      "lang": "ru",
      "actions": [
        {"key": "1", "value": "yes"},
        {"key": "2", "value": "no"}
      ]
    }
  ]
}
```

Each question can:

- Be spoken via TTS (set `text` and `lang`)
- Use a pre-recorded audio file (set the audio file ID instead of text)
- Branch to another question based on the caller's keypress (`nextQuestion`)
- Record the answer as `{question_id, value, key, time}` in the `m_PollingResults` table

Audio files

Upload a pre-recorded prompt:

```
curl -X POST http://YOUR_PBX_IP/pbxcore/api/module-dialer-manage/v1/audio \
-F "file=@welcome.wav"
```

List audio files:

```
GET /pbxcore/api/module-dialer-manage/v1/audio
```

Delete an audio file:

```
DELETE /pbxcore/api/module-dialer-manage/v1/audio/welcome.wav
```

Polling results

Fetch results incrementally (useful for syncing to your CRM):

```
GET /pbxcore/api/module-dialer-manage/v1/polling-results/0
```

The path parameter is a Unix timestamp. The endpoint returns only results newer than that timestamp. Advance your cursor to the latest result's `changeTime` after each sync.

7. Extensions setup

The **Extensions** table (on the main module page) maps internal MikoPBX extensions to survey flows. Each extension can have: - A survey to play if the caller is a NEW client (not in `m_Clients` table) - A survey to play if the caller is a KNOWN client

To add an extension: 1. Click **Add extension** on the main module page 2. Fill in: - **Extension**: e.g. `200` - **Survey if new client**: select a survey (or leave empty) - **Survey if known client**: select a survey (or leave empty) 3. Save

8. Live dashboard

Sidebar → **Dialer dashboard**

The dashboard has TWO view modes — overview and detail — both auto-refreshing every 4 seconds.

Overview mode (default)

Sidebar → **Dialer dashboard** (no campaign ID in URL)

Shows a grid of campaign cards, plus a global agent panel and recent calls feed.

Campaign cards (overview)

- One card per campaign, sorted newest first

- Each card shows:
 - Campaign name + state (open/closed/paused, color-coded)
 - Progress bar (filled based on total dialed)
 - Live in-progress count (orange badge when > 0)
 - Max channels
 - AMD enabled indicator
- Action buttons: **Details**, Edit, CSV export, Pause/Resume
- Click **Details** on any card to drill down to the detail view

Agent status panel

- One card per configured extension
- Color-coded state:
 -  Idle (ready to take calls)
 -  In call (currently bridged)
 -  Ringing (call coming in)
 -  Unavailable (offline)
- Updates every 4 seconds

Recent calls feed (overview)

- Last 20 calls across ALL campaigns
- Columns: time, number, state (ANSWER/NOANSWER/BUSY/DIAL), duration, cause

Detail mode (drill-down)

URL: `/admin-cabinet/module-auto-dialer-manage/dashboard/{id}` (or click **Details** on any campaign card)

Shows full-page detail for a SINGLE campaign:

Header card

- Campaign name, ID, CRM ID
- All status badges: state, AMD, schedule, time window, webhook
- Live statistics grid:
 - In progress (live count)
 - Max channels
 - Total dialed
 - Answered (green)
 - Failed (red)
 - Answer rate (%)

- Avg duration
- Progress bar showing answered/total ratio
- Action buttons:
 - **Resume** — set state to open (0)
 - **Pause** — set state to paused (2)
 - **Close** — set state to closed (1)
 - **Export CSV** — download all results
 - **Edit campaign** — jump to the regular edit form

Two-column layout

- **Left: Agent status panel** — same color-coded grid as overview
- **Right: Live call feed** — last 50 calls FOR THIS CAMPAIGN ONLY (filtered), with recording-lookup link per row

IVR/poll answer feed (polling campaigns only)

- Visible only when `innerNumType = polling`
- Shows: time, number, question ID, key pressed, answer value
- Last 50 answers for this campaign's survey

Auto-refresh

- Toggle on/off at top right of either view
- Default refresh interval: 4 seconds
- All data is fetched from the REST API (no WebSocket needed)
- The “Back to overview” button returns to the card grid

9. DNC blacklist

Sidebar → **DNC blacklist**

The Do-Not-Call (DNC) blacklist is a list of phone numbers that the dialer will NEVER call, regardless of which campaign they appear in. This is critical for regulatory compliance in many jurisdictions.

Adding numbers

You can add numbers in three ways:

1. **Via UI** — sidebar → DNC blacklist → enter numbers (one per line, or comma-separated), add reason, click “Add to blacklist”
2. **Via REST API** — see Developer Guide

3. **Automatically via AMD** — when AMD detects a voicemail machine, you can configure the dialer to auto-add the number to the blacklist with `source=auto-amd` (requires custom dialplan hook)

Listing and searching

- The UI shows a paginated list (50 per page) with search by partial number match
- Use the search box to find specific numbers
- Each entry shows: number, reason, source, date added, delete button

Sources

Source	Meaning
<code>manual</code>	Added by a human via UI
<code>complaint</code>	Added because the customer complained
<code>regulator</code>	Added by regulatory authority (e.g. federal DNC registry)
<code>auto-amd</code>	Auto-added by AMD when voicemail detected

Effect on dialer

When `WorkerDialer` picks up a number to dial, it first checks the blacklist. If found: - The call is NOT made - The number is marked as “skipped” in `TaskResults` with `cause=dnc` - The worker moves to the next number

This check happens BEFORE any other filter (schedule, AMD, etc.).

10. AMD (Answering Machine Detection)

AMD detects whether a call was answered by a human or a voicemail machine. This is critical for outbound dialing — you don’t want to waste agent time on voicemails.

Enabling AMD

Per-campaign setting. Either: - Set `amdEnabled: 1` in the POST `/task` body when creating a campaign - Or update an existing campaign: `PUT /task/{id}` body `{"amdEnabled": 1}`

How it works

When AMD is enabled: 1. The dialer dials the customer's number 2. When the call connects, Asterisk runs `AMD(initial_silence=2500, greeting=1500, after_greeting_silence=800, ...)` 3. AMD analyzes the audio for ~5 seconds 4. Result is one of: - `HUMAN` — proceed to bridge to agent - `MACHINE` — hang up immediately, mark as `NOANSWER` - `NOTSURE` — proceed to bridge (default fallback)

Tuning AMD parameters

The default parameters work well for Russian/English callers. To tune them, edit `Lib/AutoDialerConf.php` and find:

```
'same => n,ExecIf(["${M_AMD_ENABLED}" == "1"]?
               AMD(initial_silence=2500,greeting=1500,after_greeting_silence=800,total_analysis_time=5000,min_w
```

Parameter reference (from Asterisk docs):

Parameter	Default	Description
<code>initial_silence</code>	2500ms	Max silence before greeting
<code>greeting</code>	1500ms	Max length of greeting
<code>after_greeting_silence</code>	800ms	Silence after greeting to confirm machine
<code>total_analysis_time</code>	5000ms	Max time to analyze
<code>min_word_length</code>	100ms	Min length of a "word" (sound)
<code>between_words_silence</code>	50ms	Min silence between words
<code>maximum_number_of_words</code>	5	Max words in greeting
<code>silence_threshold</code>	256	Audio energy threshold for "silence"

Rebuild dialplan after change

After editing AMD parameters:

```
asterisk -rx 'dialplan reload'
```

11. Scheduling (business hours)

Each campaign has two scheduling fields:

Time-of-day: `timeStart` and `timeEnd`

Minutes from midnight. The dialer will only dial between these times.

Hours	timeStart	timeEnd
09:00 - 18:00	540	1080
10:00 - 20:00	600	1200
24/7 (no limit)	0	1440

Day-of-week: `scheduleDays`

Comma-separated ISO weekday numbers (1=Monday, 7=Sunday). Empty string = dial every day.

Schedule	<code>scheduleDays</code> value
Mon-Fri (workdays)	"1,2,3,4,5"
Mon-Sat	"1,2,3,4,5,6"
Weekends only	"6,7"
Every day	" " (empty)
Wed & Fri only	"3,5"

Example: Mon-Fri 09:00-18:00

```
POST /pbxcore/api/module-dialer-manage/v1/task
{
  "name": "Office hours campaign",
  "timeStart": 540,
  "timeEnd": 1080,
  "scheduleDays": "1,2,3,4,5",
  ...
}
```

How the dialer enforces this

On each iteration, `WorkerDialer` checks: 1. Is the current time between `timeStart` and `timeEnd` ? (already existed) 2. Is today's weekday in `scheduleDays` ? (new Bit Dream IT extension)

If either check fails, the worker logs `"Skipping: today not in scheduleDays"` and moves on. Numbers are NOT marked as dialed, so they'll be retried on the next valid day.

12. Webhook — get notified when a campaign state changes

Each campaign can have a `callbackUrl`. The worker fires TWO event types to that URL. Both events go to the SAME URL — your receiver should switch on the `event` field.

Event 1: `campaign.state_changed`

Fires whenever the campaign's state transitions between `open (0)`, `paused (2)`, or `closed (1)`.

Triggered by: - User clicks Resume / Pause / Close buttons on the dashboard detail view - API calls:

`PUT /task/{id}` with `state` field - Worker auto-close (when all numbers are dialed) - Auto-close from any other module hook

Payload:

```
{
  "event": "campaign.state_changed",
  "task_id": 42,
  "name": "Payment reminders - June",
  "crm_id": "1001",
  "old_state": 0,
  "new_state": 2,
  "old_state_label": "open",
  "new_state_label": "paused",
  "changed_at": "2025-07-25T15:30:00+03:00"
}
```

State value reference: - `0` = open (active, dialing) - `1` = closed (finished or stopped) - `2` = paused (temporarily on hold)

Event 2: `campaign.completed`

Fires EXACTLY ONCE when a campaign transitions to closed (1). Includes summary stats. Tracked in worker memory to guarantee exactly-once delivery — even if the worker restarts before the receiver acknowledges, the webhook will fire on the next worker iteration after restart.

Payload:

```
{
  "event": "campaign.completed",
  "task_id": 42,
  "name": "Payment reminders - June",
  "crm_id": "1001",
  "total": 1500,
  "answered": 872,
  "failed": 628,
  "completed_at": "2025-07-25T15:30:00+03:00"
}
```

Setting the webhook URL

Either at creation:

```
POST /pbxcore/api/module-dialer-manage/v1/task
{
  "name": "...",
  "callbackUrl": "https://crm.example.com/webhooks/dialer",
  ...
}
```

Or update later:

```
PUT /pbxcore/api/module-dialer-manage/v1/task/42
{"callbackUrl": "https://crm.example.com/webhooks/dialer"}
```

Webhook delivery guarantees

- The `state_changed` event fires for EVERY transition detected by the worker (which polls DB ~1/sec)
- The `completed` event fires EXACTLY ONCE per close transition. If a campaign is re-opened and then closed again, `completed` will fire again (each close = one event)
- HTTP timeout: 5 seconds
- Failed deliveries are logged but NOT retried
- The worker does NOT verify TLS certificates (so self-signed certs work)
- Both events go to the same `callbackUrl` — differentiate via the `event` field

Sample receiver (Laravel)

```
// routes/api.php
Route::post('/webhooks/dialer', function (Request $request) {
    $payload = $request->validate([
        'event' => 'required|string|in:campaign.completed,campaign.state_changed',
        'task_id' => 'required|integer',
        'name' => 'required|string',
        'crm_id' => 'nullable|string',
    ]);

    switch ($payload['event']) {
        case 'campaign.state_changed':
            $data = $request->validate([
                'old_state' => 'integer',
                'new_state' => 'integer',
                'old_state_label' => 'string',
                'new_state_label' => 'string',
                'changed_at' => 'string',
            ]);
            Log::info("Campaign {$payload['task_id']} state: {$data['old_state_label']} →
            {$data['new_state_label']}");
            // Update your CRM status, notify Slack, etc.
            break;

        case 'campaign.completed':
            $data = $request->validate([
                'total' => 'integer',
                'answered' => 'integer',
                'failed' => 'integer',
                'completed_at' => 'string',
            ]);
            Log::info("Campaign {$payload['task_id']} completed: {$data['answered']}/
            {$data['total']} answered");
            // Generate final report, send email summary, mark CRM campaign as done
            break;
    }

    return response()->json(['ok' => true]);
});
```

13. Call recordings

MikoPBX automatically records all calls if call recording is enabled system-wide. To find the recording file for a specific campaign call:

```
GET /pbxcore/api/module-dialer-manage/v1/recording/{linkedId}
```

Replace `{linkedId}` with the `linkedId` from a `TaskResults` row (returned by the `/results` endpoint).

Response

```
{
  "result": true,
  "data": {
    "linked_id": "PBX-1234567.890",
    "recording_path": "/storage/usbdisk1/mikopbx/astspool/monitor/2025/07/25/...",
    "recording_exists": true,
    "duration": 45,
    "dialstatus": "ANSWERED",
    "src": "9997912345678",
    "dst": "200",
    "calldate": "2025-07-25 15:30:00"
  }
}
```

If `recording_exists` is `false`, the recording file was deleted or never created (check MikoPBX system settings → Call recording).

14. Reports & CSV export

Campaign summary

```
GET /pbxcore/api/module-dialer-manage/v1/task/42/summary
```

Returns:

```
{
  "result": true,
  "data": {
    "task_id": 42,
    "name": "Payment reminders - June",
    "total_dialed": 1500,
    "answered": 872,
    "failed": 628,
    "answer_rate": 58.13,
    "avg_duration_sec": 47,
    "total_duration_sec": 40984,
    "state": 1,
    "state_label": "closed",
    "amd_enabled": 1
  }
}
```

CSV export

```
GET /pbxcore/api/module-dialer-manage/v1/task/42/export
```

Downloads a CSV file with columns: `number`, `state`, `duration_sec`, `attempt`, `time`, `cause`. UTF-8 with BOM (auto-detected by Excel).

Live results sync (incremental)

```
GET /pbxcore/api/module-dialer-manage/v1/results/{changeTime}
```

- `changeTime` is a Unix timestamp
 - Returns only results newer than `changeTime`
 - After processing, advance your cursor to the latest result's `changeTime`
 - Safe to poll every 3-5 seconds
-

15. Settings reference

Tasks model fields

Field	Type	Default	Description
<code>id</code>	int	auto	Primary key
<code>crmId</code>	string	-	External CRM ID
<code>name</code>	string	-	Campaign name
<code>description</code>	string	""	Optional description
<code>innerNum</code>	string	-	Extension to bridge to
<code>innerNumType</code>	string	"exten"	<code>exten</code> or <code>polling</code>
<code>maxCountChannels</code>	int	1	Max simultaneous calls
<code>maxAttempt</code>	int	1	Max retry attempts per number
<code>tryInterval</code>	int	60	Seconds between retries
<code>attemptUntilSignal</code>	int	0	If 1, retry until ANY signal (not just answer)
<code>timeStart</code>	int	0	Minutes from midnight (start time)
<code>timeEnd</code>	int	1440	Minutes from midnight (end time)
<code>scheduleDays</code>	string	""	ISO weekday numbers (1-7, comma-separated)
<code>state</code>	int	0	0=open, 1=closed, 2=paused
<code>dialPrefix</code>	string	""	Prefix prepended to outbound numbers
<code>isCallback</code>	int	0	1 = callback mode (call customer first, then bridge)
<code>amdEnabled</code>	int	0	1 = enable AMD
<code>callbackUrl</code>	string	""	Webhook URL for completion notification

Blacklist model fields

Field	Type	Description
<code>id</code>	int	Primary key
<code>number</code>	string	Phone number (digits only, unique)
<code>reason</code>	string	Why this number is blocked
<code>source</code>	string	<code>manual</code> , <code>complaint</code> , <code>regulator</code> , <code>auto-amd</code>
<code>createdAt</code>	int	Unix timestamp

16. Troubleshooting

Module won't install — “ParseError” or “Class not found”

1. Make sure you cleared the volt + opcache:

```
rm -rf /var/tmp/www_cache/volt/* /var/tmp/www_cache/translations/*
/etc/rc.d/rc.php-fpm restart
```

2. Re-install the module from the zip

Campaign is “open” but no calls are being made

Check the worker log:

```
tail -f /var/log/mikopbx/ModuleAutoDialerManage/WorkerDialer.log
```

Common reasons: - `"maxCountChannels(N) <= in_progress(M)"` — all channels are busy, waiting for one to free up - `"Number: 200, State: (BUSY) is BUSY"` — the agent extension is in use - `"Skipping blacklisted number"` — number is in DNC - `"Skipping: today not in scheduleDays"` — outside scheduled days - `"No next phone"` — all numbers in the campaign have been dialed (or are pending retry)

Worker not running at all

```
ps aux | grep WorkerDialer
```

If no process:

```
/etc/rc.d/rc.worker-safe-scripts restart
```

If still not running, start manually to see the error:

```
php
    -f /storage/usbdisk1/mikopbx/custom_modules/ModuleAutoDialerManage/bin/
    WorkerDialer.php start
```

Dashboard shows no campaigns

The dashboard fetches from `Tasks::find()`. If you see no campaigns but you created some via API, check:

```
mysql -u root mikopbxdb -e 'SELECT * FROM m_ModuleAutoDialerManage_Tasks;'
```

If the table is empty, your API calls aren't reaching the DB. Check the Apache/PHP error log:

```
tail -100 /var/log/apache2/error.log
```

Webhook not firing

1. Confirm the campaign's `callbackUrl` field is set:

```
mysql -u root mikopbxdb -e 'SELECT id, name, callbackUrl, state FROM
    m_ModuleAutoDialerManage_Tasks WHERE callbackUrl != "';'
```

2. Confirm the campaign state is `1` (closed). Webhooks only fire on closed campaigns.
3. Check the worker log for `"Fired completion webhook to ..."` entries.
4. Test your webhook URL manually:

```
curl -X POST https://your.webhook.url \
    -H "Content-Type: application/json" \
    -d '{"event": "test", "task_id": 0, "name": "manual test"}'
```

AMD is detecting humans as machines

The default AMD parameters are conservative. To make detection less aggressive: - Increase `initial_silence` (e.g. 3500) - Increase `after_greeting_silence` (e.g. 1200) - Decrease `silence_threshold` (e.g. 128 = more sensitive to noise)

After editing, reload dialplan:

```
asterisk -rx 'dialplan reload'
```

Can't access dashboard page (404)

Make sure you restarted PHP-FPM and cleared volt cache after install. The dashboard route is `module-auto-dialer-manage/dashboard` — confirm it appears in your browser URL.

17. Uninstall & rollback

Uninstall the module (keep data)

1. MikoPBX → Modules → find “ModuleAutoDialerManage” → click uninstall
2. Don't check “Delete settings” if you want to keep your campaigns

The module's tables (`m_ModuleAutoDialerManage_*`) stay in the DB. You can re-install the module later and your data will be there.

Roll back to original `ModuleAutoDialer`

If you want to go back to MIKO's original module:

1. Uninstall `ModuleAutoDialerManage` (keep data)
2. Install the original `ModuleAutoDialer` zip (from `mikopbx/ModuleAutoDialer` releases)
3. The original module will use its own tables (`m_ModuleAutoDialer` , `m_Clients` , etc.) which still contain your data from before you installed this Bit Dream IT fork

Permanently delete all module data

After uninstalling:

```
DROP TABLE IF EXISTS m_ModuleAutoDialerManage_Blacklist;
DROP TABLE IF EXISTS m_ModuleAutoDialerManage_ClientsPhones;
DROP TABLE IF EXISTS m_ModuleAutoDialerManage_ClientsProperties;
DROP TABLE IF EXISTS m_ModuleAutoDialerManage_Clients;
DROP TABLE IF EXISTS m_ModuleAutoDialerManage_TaskResults;
DROP TABLE IF EXISTS m_ModuleAutoDialerManage_PollingResults;
DROP TABLE IF EXISTS m_ModuleAutoDialerManage_QuestionActions;
DROP TABLE IF EXISTS m_ModuleAutoDialerManage_Question;
DROP TABLE IF EXISTS m_ModuleAutoDialerManage_Polling;
DROP TABLE IF EXISTS m_ModuleAutoDialerManage_Tasks;
DROP TABLE IF EXISTS m_ModuleAutoDialerManage_DialerExtensions;
DROP TABLE IF EXISTS m_ModuleAutoDialerManage_AudioFiles;
DROP TABLE IF EXISTS m_ModuleAutoDialerManage;
```

Also remove sidebar menu settings:

```
DELETE FROM m_PbxSettings WHERE `key` LIKE 'AdditionalMenuItemModuleAutoDialerManage%';
```

Support

- **Module maintainer:** Bit Dream IT (support@bitdreamit.com)
- **Upstream author:** MIKO LLC (Alexey Portnov, Nikolay Beketov)
- **License:** GPL-3.0-or-later
- **Source code:** <https://github.com/bitdreamit/ModuleAutoDialerManage>
- **Issue tracker:** <https://github.com/bitdreamit/ModuleAutoDialerManage/issues>

For bug reports, please include: 1. Module version (`cat /storage/usbdisk1/mikopbx/custom_modules/ModuleAutoDialerManage/module.json | grep version`) 2. MikoPBX version 3. Worker log excerpt (`/var/log/mikopbx/ModuleAutoDialerManage/WorkerDialer.log`) 4. Steps to reproduce